Arquitetura e evidências - Deliverable 2

Projeto HestIA | Modelagem e Construção de Software | 20/10/2026

Identificação e escopo

Versão local examinada: feature/SCRUM-531-filtros-empresa-perfil, commit base a07df833fc8b6ceef0c9c40dacf7407134a35c91, acrescida das alterações locais. A develop (a07df833fc8b6ceef0c9c40dacf7407134a35c91) foi integrada por fast-forward nesta pasta, preservando as alterações locais SCRUM-531. Os resultados abaixo são locais; o estado do CI remoto deve ser consultado no GitHub.

O escopo é a evolução das Sprints 2 e 3. As referências à Sprint 1 servem como base histórica. Os estados do Jira foram consultados em 20/10/2026 e não substituem evidências de execução.

Arquitetura adotada

O núcleo é um monólito organizado por módulos de domínio, em Java 21/Spring Boot, com camadas controller, DTO, service, model e repository. Persistência JPA e migrações Flyway usam PostgreSQL; testes de persistência usam H2. A interface usa Node/Express/EJS, com rotas que fazem a mediação com o backend. Alert Service é um processo auxiliar Node/Express acessado por HTTP. Sua existência não transforma o núcleo em uma arquitetura de microsserviços.

Módulos relevantes: empresa, departamento, usuário, autenticação, política, ferramenta, registro de uso, conformidade, auditoria, configuração, microtreinamento, gamificação, reutilizável, RAG e notificação. O otimizador Python é uma ferramenta independente; não foi comprovada uma integração por API com o MVP.

Decisão arquitetural complementar - ADR-002

Status: descrição do estado implementado e plano de evolução, sem alegação de aprovação docente.

Mantém-se a ADR-001 e a separação por domínio. O código atual contém acesso direto a repositories de outros módulos, como DepartamentoService -> EmpresaRepository. Portanto, a arquitetura implementada tem modularidade de pacotes, mas não isolamento estrito de dependências.

Decisão para esta entrega: representar explicitamente o acoplamento existente, preservando o comportamento e os testes. A regra de fachadas permanece como objetivo de evolução, não como propriedade já comprovada. Uma futura refatoração deverá introduzir contratos públicos por módulo e testes de dependência.

Justificativa: evitar um diagrama que afirme isolamento inexistente e evitar uma refatoração ampla sem critérios de aceite específicos. Consequência: a mudança de uma entidade/repository ainda pode afetar outros módulos.

Atualização C4

Contexto: colaboradores, gestores, executivos e administradores utilizam a HestIA para governança e acompanhamento de IA. O provedor de embeddings é externo.

Contêineres: interface Node/EJS -> API Spring -> PostgreSQL; API -> Alert Service; serviço de embeddings -> provedor externo. O script de otimização recebe CSV e produz resultados independentes.

Componentes: controllers -> services -> repositories -> banco; services realizam integrações por clientes HTTP. Dependências entre módulos devem aparecer como relações existentes, inclusive as que ainda não passam por fachadas.

A versão integrada contém busca vetorial/pgvector, recuperação de senha e importação de dados adicionais. As migrações, o índice vetorial e o SQL de autorização foram validados em PostgreSQL 17.11/pgvector. O provedor de embeddings foi simulado; a experiência completa do RAG com provedor real continua pendente.

Os diagramas editáveis estão em docs/architecture/c4-*-d2.mmd. O C4 anterior é preservado como histórico da Sprint 1 e recebeu indicação da revisão D2.

Implementação e rastreabilidade

Cadastro de departamento: controller, service, repository e testes; PR 32.

Registro de uso e conformidade: pacotes registro/conformidade, testes de regras e evidências históricas no PR 34.

Reutilização e gamificação: pacotes reutilizavel/gamificacao; evolução registrada no PR 53. As alterações locais SCRUM-531 acrescentam controle de recuperação e foram conciliadas com o contexto de empresa da develop.

Notificações: alert-service e cliente Java; quatro testes Node passaram nesta execução. O teste Java ponta a ponta foi executado separadamente com ALERT_E2E=true, em serviço local na porta 3107, e passou.

Estas evidências comprovam testes e presença de implementação; não constituem uma demonstração visual completa do MVP.

Evidências de TDD - US02

História: cadastrar departamento vinculado a uma empresa, rejeitando nome vazio, empresa inexistente e duplicidade.

RED: f48606acfa5f481ad97af1054bee4c0598ed8647, introdução dos cenários de teste.

GREEN: bc94f822d973d6aef8bb07a6b5dc269593c6a7b1, implementação.

REFACTOR: 96add52be00b4ee6375b4a655c75cddcbce4be4f, organização das validações.

PR: https://github.com/tec-ricardo/HestIA/pull/32

A sequência está preservada no histórico. Nesta entrega não foram reexecutados os commits históricos, para manter a pasta original e suas alterações locais intactas. Não se apresenta um log histórico reconstruído como se fosse evidência original. Os testes atuais do cadastro integram o relatório Java.

Reprodução na versão atual: ./mvnw -Dtest=DepartamentoServiceTest,DepartamentoControllerTest test.

Pipeline e validação local

Java antes da integração: 59 testes, zero falhas, zero erros, dois ignorados. O resultado da versão integrada está registrado no fechamento de validação abaixo. Integrações ignoradas: Alert Service ponta a ponta e geração real de embeddings.

Node Alert Service: quatro testes aprovados. Frontend: verificação sintática de server.js aprovada; não equivale a teste de responsividade.

Otimização: oito testes aprovados e seis cenários executados.

O workflow provisiona PostgreSQL/pgvector descartável para os testes Java, preserva relatórios mesmo em falha e publica resultados de otimização como artefatos.

Na develop remota, a execução 39442097683 falhou no job Java. Foi corrigida a duplicação de V12: a migração de staging passou a V13, mantendo V12 de recuperação de senha. A migração local de perfis passou a V14,

preservando V11 de pgvector. O teste verifica as 14 migrações e a existência das tabelas de recuperação de senha e perfis. Nenhum banco persistente foi alterado. Bancos que já aplicaram uma das versões conflitantes precisam de inspeção do histórico Flyway antes da atualização; não executar repair automático.

Evidência remota: https://github.com/tec-ricardo/HestIA/actions/runs/39442097683

Comandos e saídas locais estão em evidencias/. Os resultados novos são locais e não substituem o acompanhamento do CI no GitHub.

Inovação nas Sprints 2 e 3

Sprint 2: pesquisa de RAG e arquitetura/banco vetorial (SCRUM-288, 346, 347), marcadas como prontas; planejamento de reutilização para reduzir chamadas e tokens.

Sprint 3: documentação do fluxo (SCRUM-451) e isolamento do RAG (SCRUM-409), marcados como prontos. A develop contém evolução de pgvector (PR 70).

Pendências registradas: interface (457), integração frontend/BFF (462), BFF/RAG/pgvector (463), histórico (464) e validação completa com política corporativa (465). Essas tarefas estavam A fazer na consulta.

Hipótese a validar: recuperar conteúdo autorizado e reutilizar respostas adequadas reduz novas chamadas, custo e tempo, preservando relevância e isolamento por empresa.

Protocolo proposto: selecionar perguntas e políticas reais autorizadas; registrar fonte, empresa e perfil; executar casos com e sem conteúdo pertinente; verificar fontes, negação de acesso entre empresas, chamadas evitadas e latência; comparar com uma linha de base. Métricas futuras não são apresentadas como resultados medidos.

Critério para fechamento

A integração e a conciliação de migrações foram realizadas. Para publicação: revisar as alterações, publicar a branch e confirmar CI remoto aprovado. Os testes PostgreSQL/pgvector passaram. Para a demonstração do produto, ainda é necessário validar a interface integrada ao RAG e ao provedor real. Os documentos entregam o planejamento permitido pelo enunciado, mas não substituem validações ainda pendentes.

Fontes

https://github.com/tec-ricardo/HestIA/tree/develop

https://github.com/tec-ricardo/HestIA/pull/32

https://github.com/tec-ricardo/HestIA/pull/34

https://github.com/tec-ricardo/HestIA/pull/53

https://github.com/tec-ricardo/HestIA/pull/70

https://luccastoffano.atlassian.net/browse/SCRUM-105

https://luccastoffano.atlassian.net/browse/SCRUM-346

https://luccastoffano.atlassian.net/browse/SCRUM-465

Conciliação e operação

Os conflitos em RespostaReutilizavelService foram resolvidos mantendo EmpresaAtual e AcessoRespostaService. Um teste adicional verifica conjuntamente o isolamento de empresa e a seleção por perfil.

O build permite sobrescrever hestia.build.directory; o padrão continua target. Nesta máquina os arquivos compilados foram gerados fora do OneDrive para evitar bloqueios na limpeza. Os fontes permanecem na pasta original.

O histórico local foi preservado em stashes de recuperação. Este material acompanha a publicação na branch feature/SCRUM-531-filtros-empresa-perfil. As evidências não devem ser interpretadas como uma nova execução remota.

BuscaVetorialService agora resolve o usuário autenticado no servidor e valida atividade, empresa e perfil. O SQL filtra o perfil antes da ordenação e do LIMIT; a indexação exige permissão explícita, inclusive para ADMIN. Cinco testes PostgreSQL exercitam filtros, migrações, indexação e negações de acesso com vetores determinísticos. Os repositórios de entidades são simulados, mas o SQL e o banco são reais.

Fechamento de validação

Validação final em 20/10/2026: 78 testes Java, zero falhas, zero erros e dois ignorados. Foram executados cinco testes específicos de PostgreSQL/pgvector, além dos testes existentes em H2.

Banco descartável: PostgreSQL 17.11, imagem pgvector/pgvector:pg17, digest sha256:ac08538c6f8b9904c33c8224c5e5706dbe760aca29db1d096972b4052c22a75d. Nenhum banco corporativo ou volume persistente existente foi alterado.

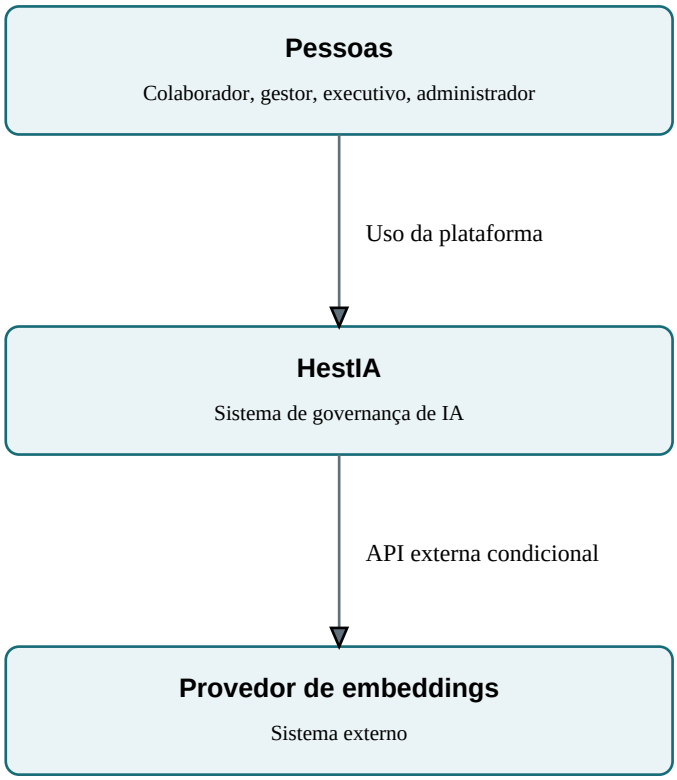
Foram verificadas 14 migrações, reaplicação sem pendências, extensão vector, índice HNSW e staging. Os testes de autorização validam filtro por perfil antes do LIMIT, empresa, usuário inativo, ADMIN sem permissão e indexação autorizada. Usam vetores determinísticos e repositórios de entidade/usuário simulados; as consultas e as migrações executam no PostgreSQL real.

O workflow CI passa a provisionar seu próprio PostgreSQL/pgvector. O resultado remoto deve ser consultado no GitHub após publicação e execução do workflow.

Os quatro testes Node, oito testes Python e seis cenários foram aprovados na validação anterior; seus fontes não mudaram nesta etapa. A integração Java -> Alert Service passou separadamente. Os dois testes ignorados na suíte completa dependem de serviço externo de notificações habilitado e credencial do provedor de embeddings.

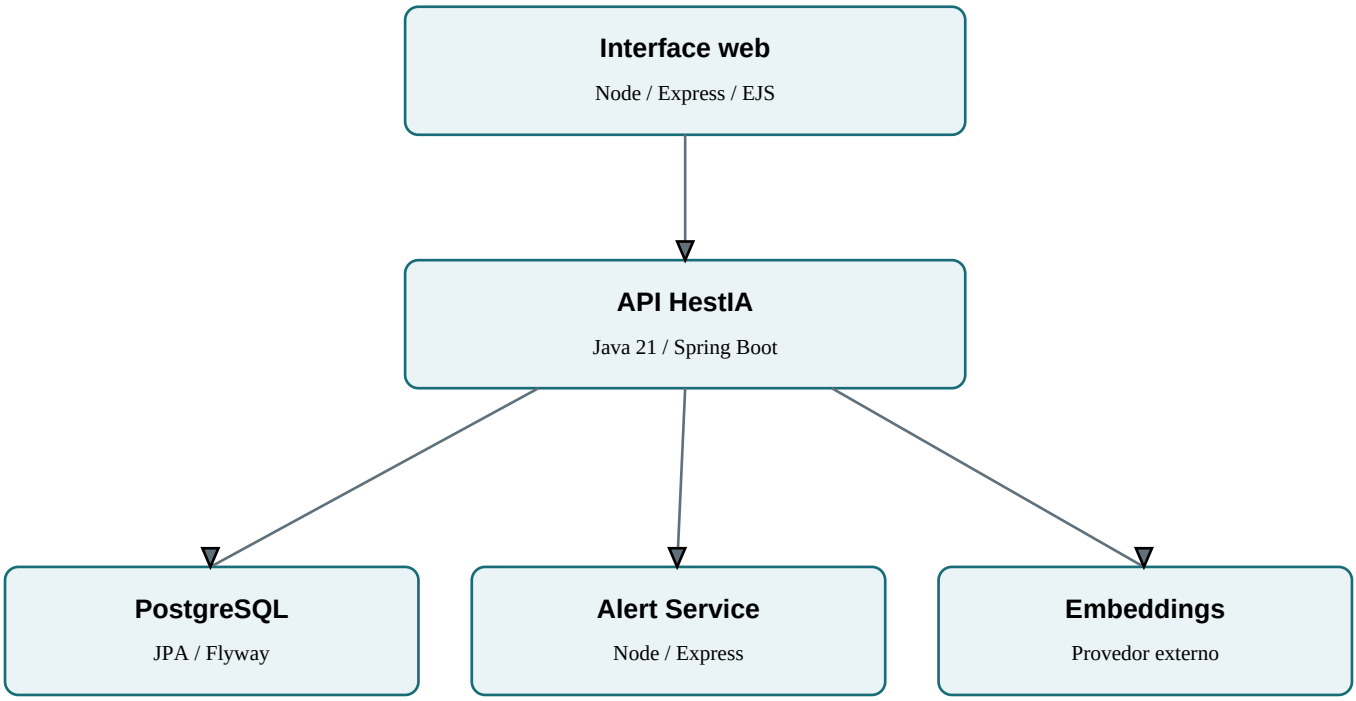
A validação completa da interface RAG com provedor real continua fora destes resultados. As evidências locais estão em evidencias/java-postgres.log.

C4 - Contexto



Visão da versão local. As integrações externas não executadas e a diferença para develop são explicitadas no relatório.

C4 - Contêineres



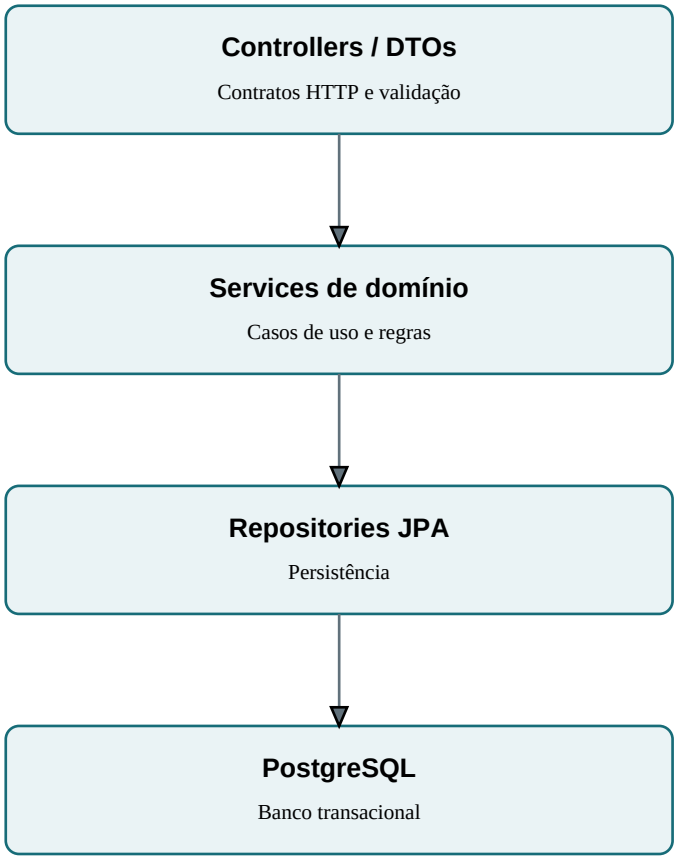
Otimizador independente

Python - CSV de entrada / JSON de saída

pgvector: integrado ao código;
SQL validado em PostgreSQL 17.11.

Visão da versão local. As integrações externas não executadas e a diferença para develop são explicitadas no relatório.

C4 - Componentes do núcleo



Acoplamento real: DepartamentoService -> EmpresaRepository.
Serviços também usam clientes HTTP; fonte Mermaid detalha as integrações.

Visão da versão local. As integrações externas não executadas e a diferença para develop são explicitadas no relatório.